Module 5: Data visualization I

PUBHLT 0411 · Python for Public Health Data Analysis

Soumik Purkayastha

Dept. of Biostatistics and Health Data Science, University of Pittsburgh

November 10, 2026

Housekeeping/setup

```
import micropip                                  # installer for packages the browser lacks
await micropip.install("seaborn")                # fetch seaborn; first run takes a few seconds

import numpy as np                               # arrays and arithmetic (Module 3)
import pandas as pd                              # tables (Module 4)
import matplotlib.pyplot as plt                  # the base plotting layer
import seaborn as sns                            # statistical figures built on matplotlib

asthma = pd.read_csv("data/pa_asthma_prevalence.csv")      # 67 counties, CDC PLACES 2024
counties = pd.read_csv("data/pa_counties.csv")             # region, urbanicity, metro, population
joined = asthma.merge(counties, on=["county", "fips"],     # attach county attributes by key
                      how="left")                          # keep every asthma row

print(joined.shape)                              # rows, columns: expect (67, 11)
```

```
(67, 11)
```

Every code cell in this deck loads its own data, so any slide runs on its own.

`micropip` is specific to Python running inside a browser. In JupyterLab, `import seaborn as sns` is sufficient — seaborn ships with the course environment.

Graphics in Python

Summarising data, numerically or graphically, is a central part of any analysis. Three layers produce figures in Python:

1. **matplotlib** — the original plotting library. Every figure in Python is ultimately a matplotlib figure.
2. **pandas** `.plot()` — a shortcut that calls matplotlib directly from a DataFrame.
3. **seaborn** — statistical figures built on matplotlib: it takes a DataFrame and column names, and handles grouping, colour, and legends.

This module begins with matplotlib, which every other layer depends on, and then turns to seaborn. Module 6 continues with seaborn's statistical layers, facets, scales, and themes.

The asthma data

```
  1  import pandas as pd                                          # tables
  2
  3  asthma = pd.read_csv("data/pa_asthma_prevalence.csv")        # one row per county
  4  print(asthma.head())                                         # first five rows
  5  print()                                                      # blank line between outputs
  6  print(asthma["asthma_prev_pct"].agg(["min", "max"]))         # the range across 67 counties
```

```
      county   fips  asthma_prev_pct  asthma_prev_adj_pct  smoking_prev_pct  \
0      Adams  42001             10.3                 10.6              15.7
1  Allegheny  42003             10.5                 10.6              14.7
2  Armstrong  42005             10.9                 11.2              19.6
3     Beaver  42007             10.6                 10.7              15.5
4    Bedford  42009             10.6                 10.8              18.2

   obesity_prev_pct
0              36.5
1              31.7
2              37.9
3              37.4
4              34.9

min     9.7
max    11.6
Name: asthma_prev_pct, dtype: float64
```

Source: CDC PLACES 2024, model-based estimates for adults, all 67 Pennsylvania counties.

Adult asthma prevalence runs from 9.7% to 11.6%. The entire state fits inside two percentage points.

The question this module opens

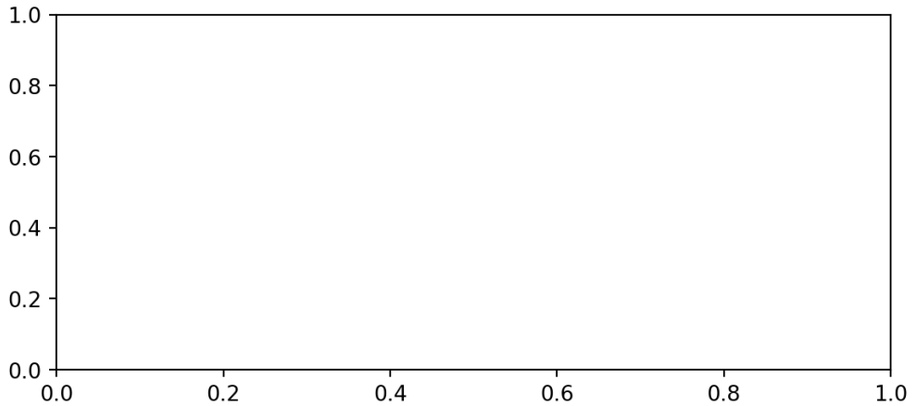
A variable that hardly varies looks like a dead end. What else moves with it?

matplotlib: the base layer

Every matplotlib figure is two objects

```
1  import matplotlib.pyplot as plt                # the plotting interface
2
3  fig, ax = plt.subplots(figsize=(7, 3))         # create a canvas and one plotting area
4
5  print(type(fig))                               # Figure: the whole canvas
6  print(type(ax))                                # Axes: one set of coordinates on it
```

```
<class 'matplotlib.figure.Figure'>
<class 'matplotlib.axes._axes.Axes'>
```



1. The **figure** is the canvas: its size, its resolution, and the file it saves to

2. The **axes** is one set of coordinates drawn on that canvas: the data, labels, and ticks

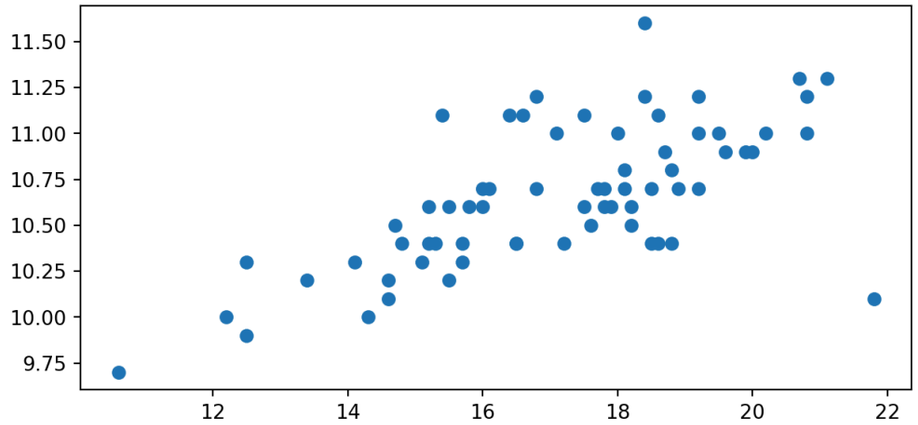
`plt.subplots()` returns both, and nearly every figure in practice begins with that call.

> **`ax` is not "axis"**
>
> An **axes** is an entire plotting area. An **axis** is one of its edges, reached as `ax.xaxis` or `ax.yaxis`. The naming is unfortunate and permanent.

Scatterplots

Examines the relationship between two numeric variables.

```
 1  import pandas as pd                                    # tables
 2  import matplotlib.pyplot as plt                        # plotting
 3
 4  asthma = pd.read_csv("data/pa_asthma_prevalence.csv")  # one row per county
 5
 6  fig, ax = plt.subplots(figsize=(6.5, 3.2))             # canvas and one axes
 7  ax.scatter(x=asthma["smoking_prev_pct"],               # horizontal position: smoking
 8             y=asthma["asthma_prev_pct"])                # vertical position: asthma
 9  plt.tight_layout()                                     # keep labels inside the canvas
10  plt.show()                                             # render the figure
```



matplotlib receives **columns**, not column names. A bare name is an undefined variable:

```
 1  import matplotlib.pyplot as plt                # plotting
 2
 3  fig, ax = plt.subplots(figsize=(4, 2))         # canvas and one axes
 4  ax.scatter(x=smoking_prev_pct,                 # a bare name: Python looks for a variable
 5             y=asthma_prev_pct)                  # NameError: name 'smoking_prev_pct' is not define
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[213], line 4
      1 import matplotlib.pyplot as plt                # plotting
      2
      3 fig, ax = plt.subplots(figsize=(4, 2))         # canvas and one axes
----> 4 ax.scatter(x=smoking_prev_pct,                 # a bare name: Python looks for a
variable
      5            y=asthma_prev_pct)                  # NameError: name
'smoking_prev_pct' is not defined

NameError: name 'smoking_prev_pct' is not defined
```

Histograms

A graphical summary of the distribution of one numeric variable. `bins` sets the number of bins, or their exact edges.

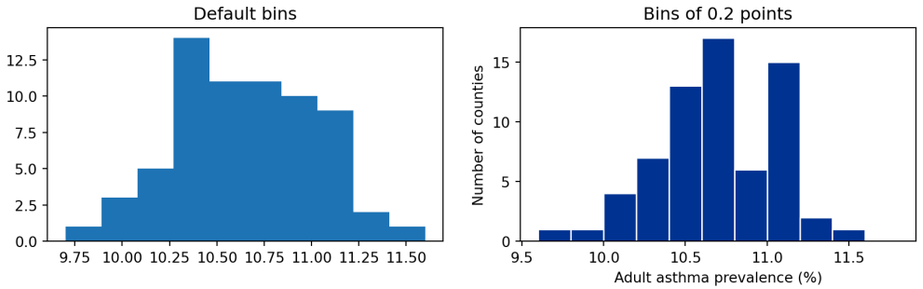
```
import numpy as np                                      # evenly spaced bin edges
import pandas as pd                                     # tables
import matplotlib.pyplot as plt                         # plotting

asthma = pd.read_csv("data/pa_asthma_prevalence.csv")   # one row per county
prev = asthma["asthma_prev_pct"]                        # the column to summarise

fig, (left, right) = plt.subplots(1, 2, figsize=(9, 3))   # two panels side by side
left.hist(prev)                                         # default: 10 bins, edges set by the dat
left.set_title("Default bins")                          # panel title

edges = np.linspace(9.6, 11.8, 12)                      # 11 bins of width 0.2, on round numbers
right.hist(prev, bins=edges,                            # explicit bin edges
           color="#003594", edgecolor="white")          # fill colour; white gaps between bars
right.set_title("Bins of 0.2 points")                   # panel title
right.set_xlabel("Adult asthma prevalence (%)")         # x-axis label with units
right.set_ylabel("Number of counties")                  # y-axis label

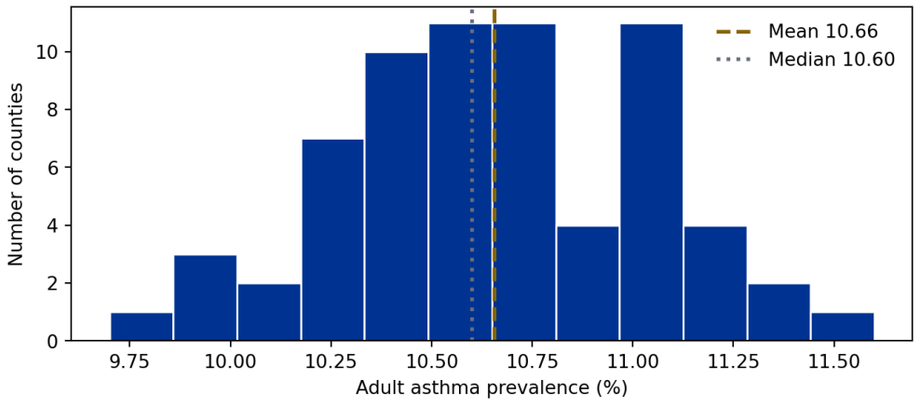
plt.tight_layout()                                      # stop panels overlapping
plt.show()                                              # render the figure
```



Histograms + reference lines

A second layer drawn on the same axes. `ax.axvline()` adds a vertical line at a value.

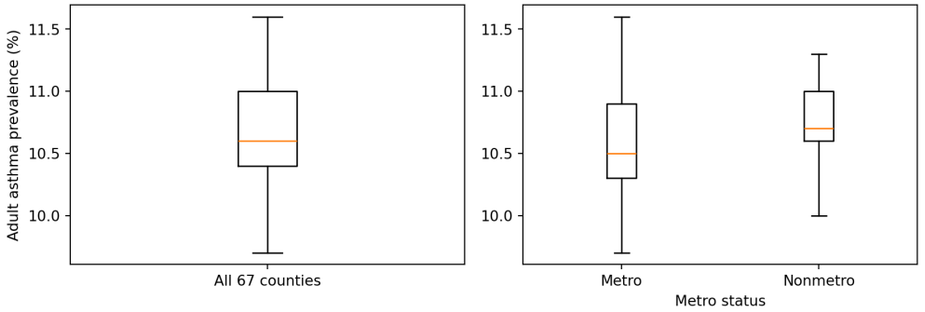
```
 1  import pandas as pd                                          # tables
 2  import matplotlib.pyplot as plt                              # plotting
 3
 4  asthma = pd.read_csv("data/pa_asthma_prevalence.csv")        # one row per county
 5  prev = asthma["asthma_prev_pct"]                             # the column to summarise
 6
 7  fig, ax = plt.subplots(figsize=(7, 3.2))                     # canvas and one axes
 8  ax.hist(prev, bins=12,                                       # layer 1: the histogram
 9          color="#003594", edgecolor="white")                  # fill and bar edges
10  ax.axvline(prev.mean(), color="#8a6400",                     # layer 2: a line at the mean
11             linestyle="--", linewidth=2,                      # dashed, thicker than default
12             label=f"Mean {prev.mean():.2f}")                  # legend text
13  ax.axvline(prev.median(), color="#6b7280",                   # layer 3: a line at the median
14             linestyle=":", linewidth=2,                       # dotted, so it differs from the mean
15             label=f"Median {prev.median():.2f}")              # legend text
16  ax.set_xlabel("Adult asthma prevalence (%)")                 # x-axis label
17  ax.set_ylabel("Number of counties")                          # y-axis label
18  ax.legend(frameon=False)                                     # legend from each layer's label=
19  plt.tight_layout()                                           # keep labels inside the canvas
20  plt.show()                                                   # render the figure
```



Boxplots

A graphical summary of a distribution: the **median**, the **interquartile range** (the box), and the spread beyond it (the whiskers).

```
 1  import pandas as pd                                          # tables
 2  import matplotlib.pyplot as plt                              # plotting
 3
 4  asthma = pd.read_csv("data/pa_asthma_prevalence.csv")        # one row per county
 5  counties = pd.read_csv("data/pa_counties.csv")               # metro status per county
 6  joined = asthma.merge(counties, on=["county", "fips"])       # one table
 7
 8  levels = ["Metro", "Nonmetro"]                               # the two groups, in display order
 9  groups = [joined.loc[joined["metro"] == m,                   # rows in group m ...
10                       "asthma_prev_pct"]                     # ... asthma column only
11            for m in levels]                                  # one Series per group
12
13  fig, (one, by) = plt.subplots(1, 2, figsize=(9, 3.2))        # two panels
14  one.boxplot(joined["asthma_prev_pct"])                       # a single distribution
15  one.set_xticks([1], ["All 67 counties"])                     # boxes sit at x = 1, 2, ...
16  one.set_ylabel("Adult asthma prevalence (%)")                # y-axis label
17
18  by.boxplot(groups)                                           # one box per Series in the list
19  by.set_xticks([1, 2], levels)                                # name the boxes
20  by.set_xlabel("Metro status")                                # x-axis label
21
22  plt.tight_layout()                                           # stop panels overlapping
23  plt.show()                                                   # render the figure
```



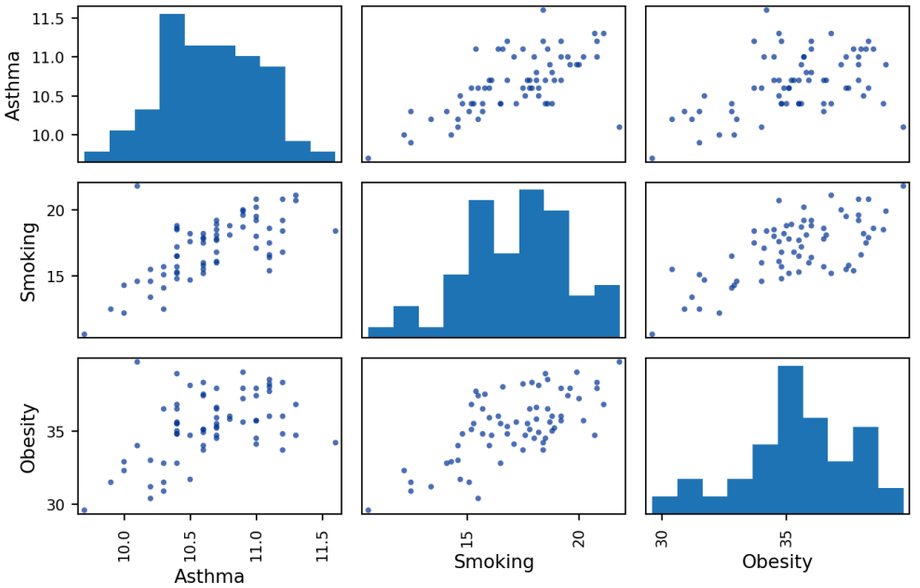
Pairs plots

`pd.plotting.scatter_matrix()` draws every pairwise scatter among several numeric columns, with a histogram of each column on the diagonal.

```
import pandas as pd                                          # tables
import matplotlib.pyplot as plt                              # plotting

asthma = pd.read_csv("data/pa_asthma_prevalence.csv")        # one row per county
short = asthma.rename(columns={                              # short names fit the panel edges
    "asthma_prev_pct": "Asthma",                             # adult asthma (%)
    "smoking_prev_pct": "Smoking",                           # adult smoking (%)
    "obesity_prev_pct": "Obesity"})                          # adult obesity (%)

pd.plotting.scatter_matrix(short[["Asthma", "Smoking", "Obesity"]],  # three columns, 3 x 3 grid
                           figsize=(7, 4.6),                 # canvas size in inches
                           diagonal="hist",                  # histograms on the diagonal
                           color="#003594", alpha=0.7)       # point colour and transparency
plt.tight_layout()                                           # keep labels inside the canvas
plt.show()                                                   # render the figure
```



Most useful for a handful of numeric columns.

seaborn and the grammar of graphics

The grammar of graphics and seaborn

A statistical figure is assembled from independent components (Wilkinson, 2005):

1. **Data** — a table, one row per observation

2. **Mappings** — which column drives which visual property: position, colour, shape, size

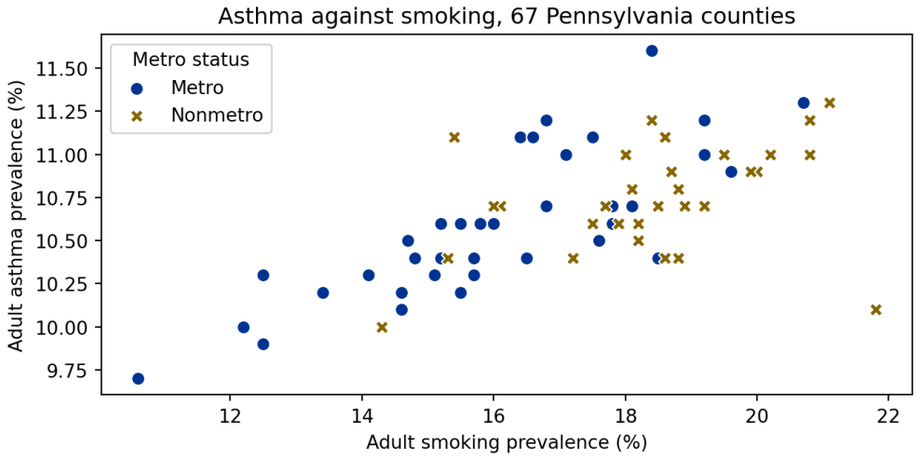
3. **A mark** — the geometric object that represents each row: a point, a line, a bar

In seaborn, the **function name is the mark** (`scatterplot`, `lineplot`, `countplot`), `data=` names the table, and every mapping is a keyword that takes a **column name**.

```
sns.scatterplot(data=joined,                # data
                x="smoking_prev_pct",       # mapping: column -> horizontal position
                y="asthma_prev_pct",        # mapping: column -> vertical position
                hue="metro")                # mapping: column -> colour
```

Goal

Build the following figure with seaborn, one component at a time.



First: an ordered categorical column

pandas sorts text alphabetically. When categories have a natural order, `pd.Categorical()` fixes it, and every figure and table then follows that order.

```
import pandas as pd                                          # tables

counties = pd.read_csv("data/pa_counties.csv")               # one row per county

print(sorted(counties["urbanicity"].unique()))               # default: alphabetical order

levels = (counties.sort_values("urbanicity_code")            # code 1 = most urban, 6 = most rural
                  ["urbanicity"].unique())                   # names in that order
counties["urbanicity"] = pd.Categorical(                     # convert text to a categorical
    counties["urbanicity"],                                  # the column to convert
    categories=levels,                                       # the order to keep
    ordered=True)                                            # the order carries meaning

print(counties["urbanicity"].cat.categories.tolist())     # the stored order
```

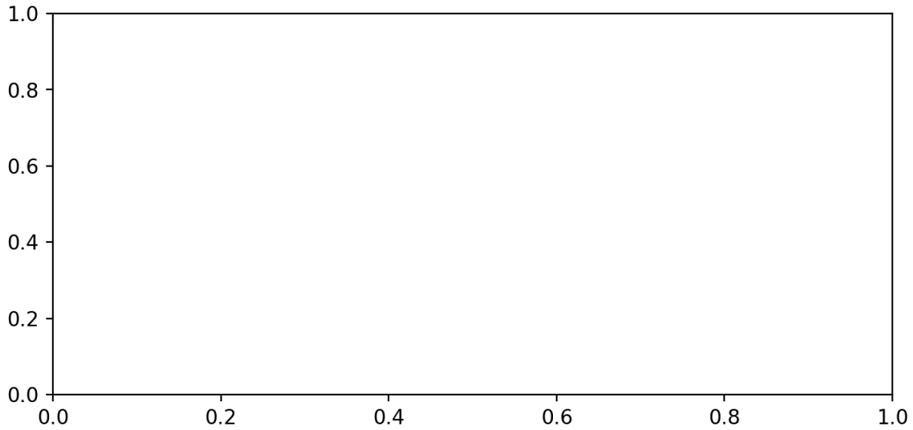
```
['Large central metro', 'Large fringe metro', 'Medium metro', 'Micropolitan', 'Noncore', 'Small metro']
['Large central metro', 'Large fringe metro', 'Medium metro', 'Small metro', 'Micropolitan', 'Noncore']
```

Text ↔ Plot (1 of 5)

Start with an **empty axes**.

```
import matplotlib.pyplot as plt                 # figure and axes

fig, ax = plt.subplots(figsize=(6.5, 3.2))      # a canvas with one empty plotting area
plt.tight_layout()                                          # keep labels inside the canvas
plt.show()                                      # render it: nothing is mapped yet
```



Every figure starts here. The axes is primed to receive data, and nothing is drawn yet.

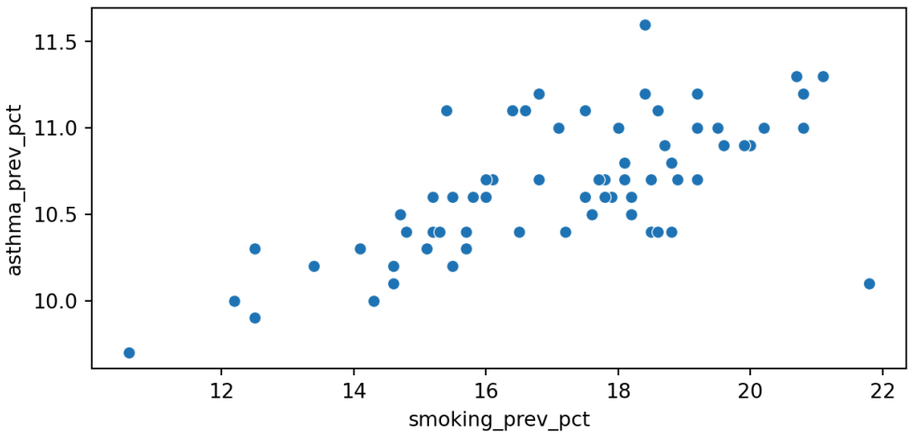
Text ↔ Plot (2 of 5)

Start with an empty axes. **Using `joined`, map smoking to the x-axis and asthma to the y-axis. Represent each county with a point.**

```
import micropip                                          # installer for browser packages
await micropip.install("seaborn")                        # fetch seaborn
import pandas as pd                                      # tables
import seaborn as sns                                    # statistical figures
import matplotlib.pyplot as plt                          # figure and axes

asthma = pd.read_csv("data/pa_asthma_prevalence.csv")    # one row per county
counties = pd.read_csv("data/pa_counties.csv")           # metro status per county
joined = asthma.merge(counties, on=["county", "fips"])   # one table

fig, ax = plt.subplots(figsize=(6.5, 3.2))               # empty axes
sns.scatterplot(data=joined,                             # data
                x="smoking_prev_pct",                    # mapping: smoking -> x
                y="asthma_prev_pct",                     # mapping: asthma -> y
                ax=ax)                                   # draw on this axes
plt.tight_layout()                                       # keep labels inside the canvas
plt.show()                                               # render the figure
```



`scatterplot` is the mark. `ax=ax` places it on the axes created above.

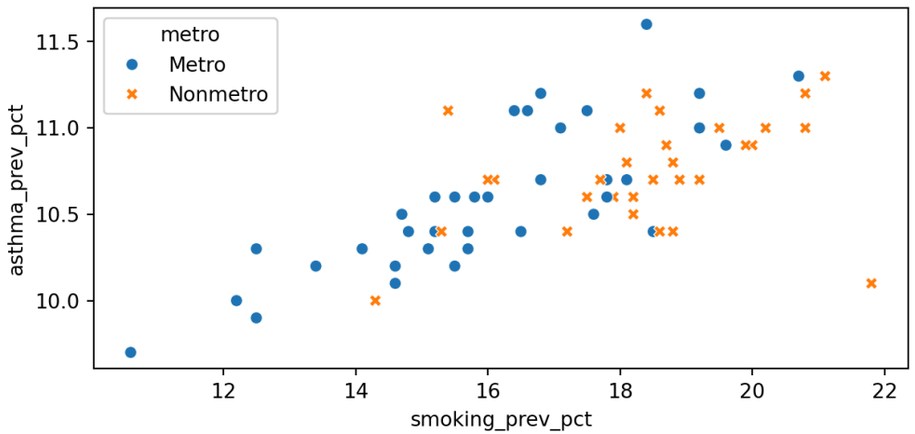
Text ↔ Plot (3 of 5)

… Represent each county with a point. **Map metro status to the colour and the shape of each point.**

```
import micropip                                          # installer for browser packages
await micropip.install("seaborn")                        # fetch seaborn
import pandas as pd                                      # tables
import seaborn as sns                                    # statistical figures
import matplotlib.pyplot as plt                          # figure and axes

asthma = pd.read_csv("data/pa_asthma_prevalence.csv")    # one row per county
counties = pd.read_csv("data/pa_counties.csv")           # metro status per county
joined = asthma.merge(counties, on=["county", "fips"])   # one table

fig, ax = plt.subplots(figsize=(6.5, 3.2))               # empty axes
sns.scatterplot(data=joined,                             # data
                x="smoking_prev_pct",                    # mapping: smoking -> x
                y="asthma_prev_pct",                     # mapping: asthma -> y
                hue="metro",                             # mapping: metro -> colour
                style="metro",                           # mapping: metro -> marker shape
                ax=ax)                                   # draw on this axes
plt.tight_layout()                                       # keep labels inside the canvas
plt.show()                                               # render the figure
```

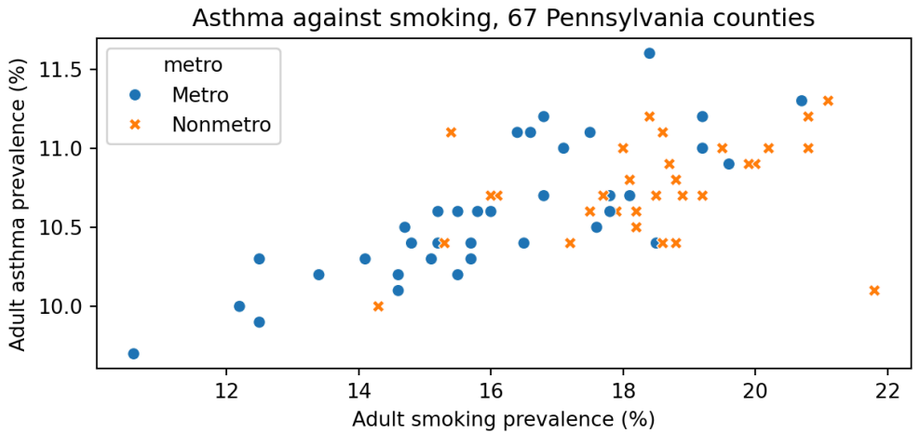


`hue=` maps a column to colour; `style=` maps a column to marker shape. seaborn builds the legend.

Text ↔ Plot (4 of 5)

… Map metro status to the colour and shape of each point. **Title the figure and label both axes with units.**

```
 1  import micropip                                          # installer for browser packages
 2  await micropip.install("seaborn")                        # fetch seaborn
 3  import pandas as pd                                      # tables
 4  import seaborn as sns                                    # statistical figures
 5  import matplotlib.pyplot as plt                          # figure and axes
 6
 7  asthma = pd.read_csv("data/pa_asthma_prevalence.csv")    # one row per county
 8  counties = pd.read_csv("data/pa_counties.csv")           # metro status per county
 9  joined = asthma.merge(counties, on=["county", "fips"])   # one table
10
11  fig, ax = plt.subplots(figsize=(6.5, 3.2))               # empty axes
12  sns.scatterplot(data=joined, x="smoking_prev_pct",       # data; smoking -> x
13                  y="asthma_prev_pct",                     # asthma -> y
14                  hue="metro", style="metro",              # metro -> colour and shape
15                  ax=ax)                                   # draw on this axes
16  ax.set(title="Asthma against smoking, 67 Pennsylvania counties",   # title
17         xlabel="Adult smoking prevalence (%)",            # x-axis label with units
18         ylabel="Adult asthma prevalence (%)")             # y-axis label with units
19  plt.tight_layout()                                       # keep labels inside the canvas
20  plt.show()                                               # render the figure
```



`ax.set()` sets several properties of the axes in one call.

The minimum for any figure that leaves a notebook

An x-label with units, a y-label with units, and a title that states what is plotted.

Text ↔ Plot (5 of 5)

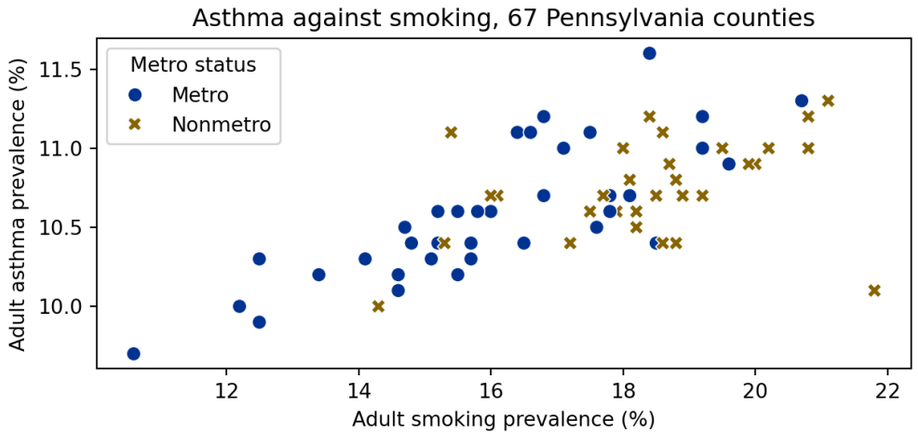
… Title the figure and label both axes with units. **Replace the default colours and title the legend.**

```
import micropip                                          # installer for browser packages
await micropip.install("seaborn")                        # fetch seaborn
import pandas as pd                                      # tables
import seaborn as sns                                    # statistical figures
import matplotlib.pyplot as plt                          # figure and axes

asthma = pd.read_csv("data/pa_asthma_prevalence.csv")    # one row per county
counties = pd.read_csv("data/pa_counties.csv")           # metro status per county
joined = asthma.merge(counties, on=["county", "fips"])   # one table

palette = {"Metro": "#003594",                           # one colour per group, by name
           "Nonmetro": "#8a6400"}                        # dark gold for nonmetro

fig, ax = plt.subplots(figsize=(6.5, 3.2))               # empty axes
sns.scatterplot(data=joined, x="smoking_prev_pct",       # data; smoking -> x
                y="asthma_prev_pct",                     # asthma -> y
                hue="metro", style="metro",              # metro -> colour and shape
                palette=palette,                         # colours for the hue groups
                s=50, ax=ax)                             # point size; target axes
ax.set(title="Asthma against smoking, 67 Pennsylvania counties",   # title
       xlabel="Adult smoking prevalence (%)",            # x-axis label with units
       ylabel="Adult asthma prevalence (%)")             # y-axis label with units
ax.get_legend().set_title("Metro status")                # replace the column-name title
plt.tight_layout()                                       # keep labels inside the canvas
plt.show()                                               # render the figure
```



This is the goal figure.

seaborn calls

`data` is seaborn's first argument and its name can be omitted. pandas `.plot.scatter()` draws the same mark from the DataFrame itself.

```
import micropip                                          # installer for browser packages
await micropip.install("seaborn")                        # fetch seaborn
import pandas as pd                                      # tables
import seaborn as sns                                    # statistical figures
import matplotlib.pyplot as plt                          # figure and axes

asthma = pd.read_csv("data/pa_asthma_prevalence.csv")    # one row per county

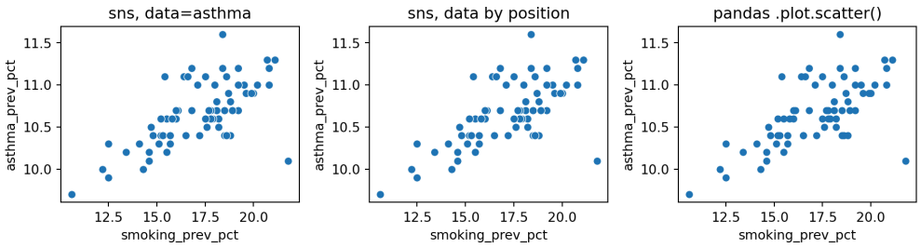
fig, (a, b, c) = plt.subplots(1, 3, figsize=(10, 2.8))   # three panels

sns.scatterplot(data=asthma, x="smoking_prev_pct",       # data named explicitly
                y="asthma_prev_pct", ax=a)               # mappings; first panel
a.set_title("sns, data=asthma")                          # panel title

sns.scatterplot(asthma, x="smoking_prev_pct",            # data passed by position
                y="asthma_prev_pct", ax=b)               # mappings; second panel
b.set_title("sns, data by position")                     # panel title

asthma.plot.scatter(x="smoking_prev_pct",                # pandas: the DataFrame draws itself
                    y="asthma_prev_pct", ax=c)           # same columns; third panel
c.set_title("pandas .plot.scatter()")                    # panel title

plt.tight_layout()                                       # stop panels overlapping
plt.show()                                               # render the figure
```



The mappings `x=` and `y=` must always be named.

Mapping versus setting

Drives the property from	a **column** in the data	a **constant**
Passed as	a column name, to `x`, `y`, `hue`, `style`, `size`	a value, to `color`, `marker`, `s`, `alpha`, `linewidth`, `linestyle`
Example	`hue="metro"`	`color="#003594"`
Legend	built automatically	none

Mappings are seaborn arguments. Settings are matplotlib arguments, which seaborn passes through to every mark.

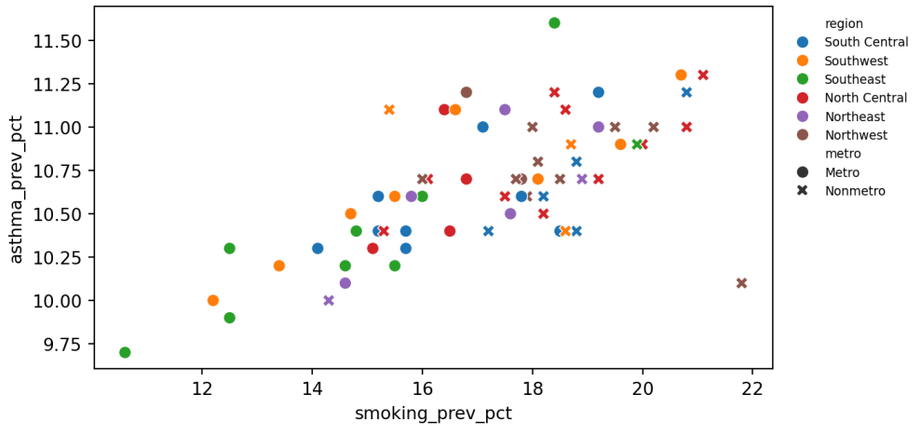
Colour and shape

Both mapped to columns: colour to region, shape to metro status.

```
import micropip                                          # installer for browser packages
await micropip.install("seaborn")                        # fetch seaborn
import pandas as pd                                      # tables
import seaborn as sns                                    # statistical figures
import matplotlib.pyplot as plt                          # figure and axes

asthma = pd.read_csv("data/pa_asthma_prevalence.csv")    # one row per county
counties = pd.read_csv("data/pa_counties.csv")           # region and metro per county
joined = asthma.merge(counties, on=["county", "fips"])   # one table

fig, ax = plt.subplots(figsize=(7.5, 3.6))               # canvas and one axes
sns.scatterplot(data=joined, x="smoking_prev_pct",       # data; smoking -> x
                y="asthma_prev_pct",                     # asthma -> y
                hue="region",                            # mapping: region -> colour
                style="metro",                           # mapping: metro -> shape
                s=50, ax=ax)                             # point size; target axes
ax.legend(fontsize=7, bbox_to_anchor=(1.01, 1),          # move the legend outside ...
          loc="upper left", frameon=False)               # ... to the right of the axes
plt.tight_layout()                                       # make room for the legend
plt.show()                                               # render the figure
```



Size

Mapped to a column (left), and set to a constant (right).

```
import micropip                                          # installer for browser packages
await micropip.install("seaborn")                        # fetch seaborn
import pandas as pd                                      # tables
import seaborn as sns                                    # statistical figures
import matplotlib.pyplot as plt                          # figure and axes

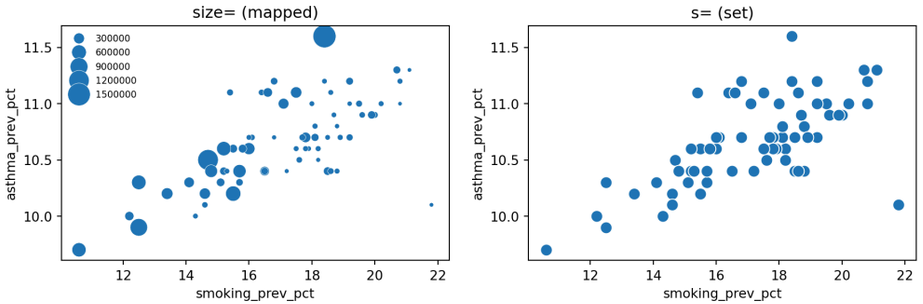
asthma = pd.read_csv("data/pa_asthma_prevalence.csv")    # one row per county
counties = pd.read_csv("data/pa_counties.csv")           # population per county
joined = asthma.merge(counties, on=["county", "fips"])   # one table

fig, (mapped, fixed) = plt.subplots(1, 2, figsize=(10, 3.4))   # two panels

sns.scatterplot(data=joined, x="smoking_prev_pct",       # data; smoking -> x
                y="asthma_prev_pct",                     # asthma -> y
                size="population_2023",                  # MAPPING: population -> marker size
                sizes=(10, 300),                         # smallest and largest marker area
                ax=mapped)                               # left panel
mapped.legend(fontsize=7, frameon=False)                 # compact legend
mapped.set_title("size= (mapped)")                       # panel title

sns.scatterplot(data=joined, x="smoking_prev_pct",       # same data and positions
                y="asthma_prev_pct",                     # asthma -> y
                s=80,                                    # SETTING: every marker the same area
                ax=fixed)                                # right panel
fixed.set_title("s= (set)")                              # panel title

plt.tight_layout()                                       # stop panels overlapping
plt.show()                                               # render the figure
```



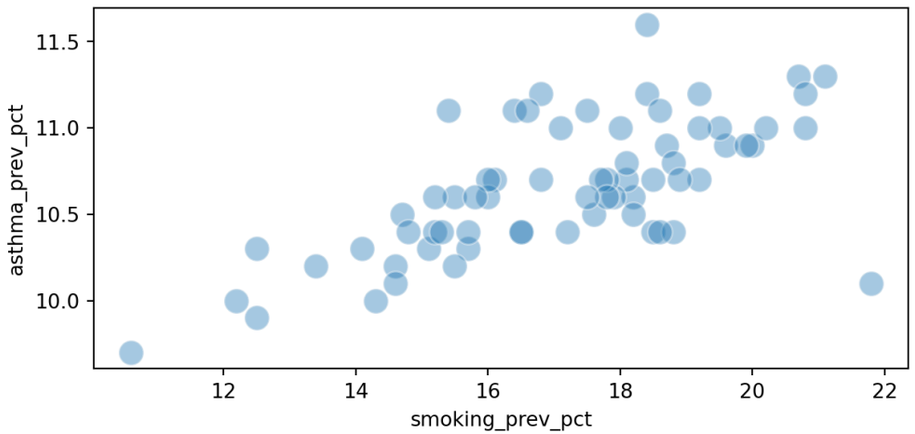
Transparency

Set to a constant between 0 and 1. seaborn has **no mapping** for transparency.

```
import micropip                                                 # installer for browser packages
await micropip.install("seaborn")                               # fetch seaborn
import pandas as pd                                             # tables
import seaborn as sns                                           # statistical figures
import matplotlib.pyplot as plt                                 # figure and axes

asthma = pd.read_csv("data/pa_asthma_prevalence.csv")           # one row per county

fig, ax = plt.subplots(figsize=(6.5, 3.2))                      # canvas and one axes
sns.scatterplot(data=asthma, x="smoking_prev_pct",              # data; smoking -> x
                y="asthma_prev_pct",                            # asthma -> y
                s=150,                                          # large markers, so overlaps show
                alpha=0.4,                                      # SETTING: 40% opaque
                ax=ax)                                          # target axes
plt.tight_layout()                                              # keep labels inside the canvas
plt.show()                                                      # render the figure
```



Passing a column name to a setting is an error:

```
sns.scatterplot(data=asthma, x="smoking_prev_pct", y="asthma_prev_pct",  # data and positions
                alpha="metro")      # TypeError: alpha must be numeric or None, not a string
```

Linetype

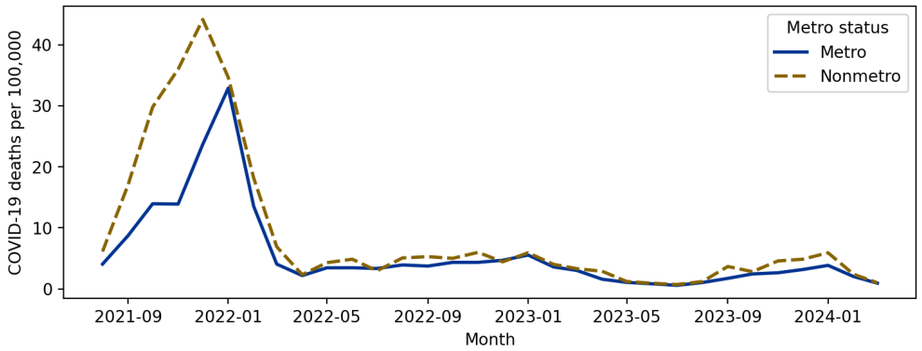
`sns.lineplot()` maps `style=` to the **dash pattern** of each line.

```
import micropip                                          # installer for browser packages
await micropip.install("seaborn")                        # fetch seaborn
import pandas as pd                                      # tables
import seaborn as sns                                    # statistical figures
import matplotlib.pyplot as plt                          # figure and axes

covid = pd.read_csv("data/pa_covid_monthly.csv")         # deaths by county and month
counties = pd.read_csv("data/pa_counties.csv")           # population and metro per county
covid["month"] = pd.to_datetime(covid["month"])          # text -> real dates
covid = covid[covid["month"] >= "2021-08-01"]            # keep the reliably reported period

merged = covid.merge(counties[["county", "population_2023", "metro"]],  # attach metro and
                     on="county")                                       # population by county
monthly = (merged.groupby(["month", "metro"], as_index=False)           # one row per month x group
                 [["covid_deaths", "population_2023"]].sum())           # total deaths and people
monthly["per_100k"] = (monthly["covid_deaths"]           # deaths ...
                       / monthly["population_2023"]      # ... per resident ...
                       * 100000)                         # ... per 100,000

fig, ax = plt.subplots(figsize=(8, 3.2))                 # canvas and one axes
sns.lineplot(data=monthly, x="month", y="per_100k",      # data; time -> x, rate -> y
             hue="metro", style="metro",                 # metro -> colour and dash pattern
             palette={"Metro": "#003594",                # course colours, by group
                      "Nonmetro": "#8a6400"},            # dark gold for nonmetro
             linewidth=2, ax=ax)                         # SETTING: line width; target axes
ax.set(xlabel="Month",                                   # x-axis label
       ylabel="COVID-19 deaths per 100,000")             # y-axis label with units
ax.get_legend().set_title("Metro status")                # legend title
plt.tight_layout()                                       # keep labels inside the canvas
plt.show()                                               # render the figure
```



Default aesthetics

Once a column is mapped, seaborn chooses the rest:

1. **Axis labels** — the column names: `smoking_prev_pct`
2. **Legend title** — the column name: `metro`
3. **Colours** — the `deep` palette, assigned in the order the groups first appear in the data
4. **Markers and dashes** — a fixed sequence: circle, cross, diamond, …
5. **Category order** — order of appearance, unless the column is a `pd.Categorical`

Each default has an override: `ax.set()`, `.set_title()` on the legend, `palette=`, `markers=`, `order=` or a categorical column.

Choosing a figure

The choice of figure follows from two questions: **what type is each variable**, and **how many variables** are shown.

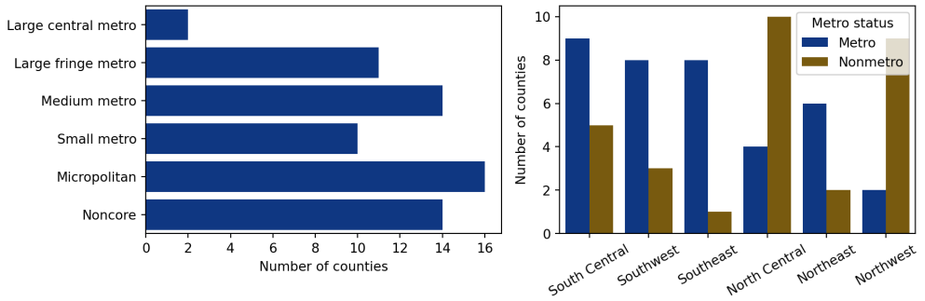
Variables	Figure	seaborn	matplotlib
One categorical	Bar of counts	`countplot`	`ax.bar`
One numeric	Histogram, boxplot	`histplot`, `boxplot`	`ax.hist`, `ax.boxplot`
Numeric by categorical	Boxplot per group	`boxplot`	`ax.boxplot`
Two categorical	Grouped bar of counts	`countplot(hue=)`	—
Two numeric	Scatter	`scatterplot`	`ax.scatter`
Numeric over time	Line	`lineplot`	`ax.plot`

A line asserts that consecutive points are **connected**. Categories are never joined by a line.

`sns.countplot()`: one and two variables

The height of each bar is the **number of rows** in each category.

```
 1  import micropip                                          # installer for browser packages
 2  await micropip.install("seaborn")                        # fetch seaborn
 3  import pandas as pd                                      # tables
 4  import seaborn as sns                                    # statistical figures
 5  import matplotlib.pyplot as plt                          # figure and axes
 6
 7  counties = pd.read_csv("data/pa_counties.csv")           # one row per county
 8  levels = (counties.sort_values("urbanicity_code")        # most urban to most rural
 9                    ["urbanicity"].unique())               # names in that order
10  counties["urbanicity"] = pd.Categorical(                 # fix the display order
11      counties["urbanicity"], categories=levels, ordered=True)  # column, order, ordered
12
13  fig, (one, two) = plt.subplots(1, 2, figsize=(10, 3.4))  # two panels
14
15  sns.countplot(data=counties, y="urbanicity",             # one variable, bars run sideways
16                color="#003594", ax=one)                   # SETTING: one colour for all bars
17  one.set(xlabel="Number of counties", ylabel="")          # labels
18
19  sns.countplot(data=counties, x="region",                 # bars per region ...
20                hue="metro",                               # ... split by metro status
21                palette={"Metro": "#003594",               # course colours, by group
22                         "Nonmetro": "#8a6400"},           # dark gold for nonmetro
23                ax=two)                                    # right panel
24  two.set(xlabel="", ylabel="Number of counties")          # labels
25  two.tick_params(axis="x", labelrotation=30)              # tilt long region names
26  two.get_legend().set_title("Metro status")               # legend title
27
28  plt.tight_layout()                                       # stop panels overlapping
29  plt.show()                                               # render the figure
```



A truncated axis manufactures a difference

```python
import pandas as pd                                          # tables
import matplotlib.pyplot as plt                              # plotting

asthma = pd.read_csv("data/pa_asthma_prevalence.csv")        # one row per county
top5 = asthma.nlargest(5, "asthma_prev_pct")                 # the five highest rates

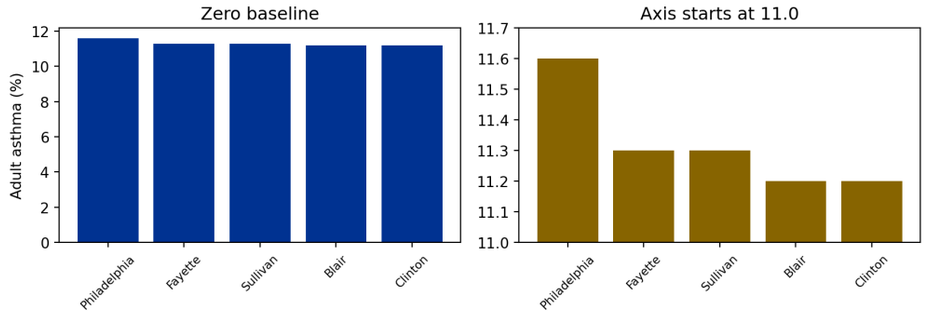
fig, (honest, cut) = plt.subplots(1, 2, figsize=(9, 3.2))    # two panels

honest.bar(top5["county"], top5["asthma_prev_pct"],          # bar per county, height = rate
           color="#003594")                                  # fill colour
honest.set_title("Zero baseline")                            # panel title
honest.set_ylabel("Adult asthma (%)")                        # y-axis label

cut.bar(top5["county"], top5["asthma_prev_pct"],             # the same five bars
        color="#8a6400")                                     # fill colour
cut.set_ylim(11.0, 11.7)                                     # start the axis at 11.0, not 0
cut.set_title("Axis starts at 11.0")                         # panel title

for panel in (honest, cut):                                  # both panels:
    panel.tick_params(axis="x", labelrotation=45,            # tilt county names
                      labelsize=8)                           # and shrink them

plt.tight_layout()                                           # stop panels overlapping
plt.show()                                                   # render the figure
```



Identical data. The right panel implies Philadelphia has three times the asthma burden of Clinton County. The true difference is 0.4 percentage points.

> **The rule for bars**
>
> A bar encodes value by **length**, so its baseline must be zero. Truncating the axis breaks the correspondence between length and value.

Describing what the figure shows

`.describe()` summarises every numeric column at once

```
import pandas as pd                                          # tables

asthma = pd.read_csv("data/pa_asthma_prevalence.csv")        # one row per county

cols = ["asthma_prev_pct", "smoking_prev_pct",               # the three measures
        "obesity_prev_pct"]                                  # ... and adult obesity (%)
print(asthma[cols].describe().round(2))                      # eight summaries per column
```

```
       asthma_prev_pct  smoking_prev_pct  obesity_prev_pct
count            67.00             67.00             67.00
mean             10.66             17.12             35.36
std               0.38              2.32              2.26
min               9.70             10.60             29.60
25%              10.40             15.50             34.15
50%              10.60             17.50             35.50
75%              11.00             18.65             37.00
max              11.60             21.80             39.70
```

Eight numbers per column: the count, the mean, the standard deviation, the minimum, the three quartiles, and the maximum.

`count` is the first number to read

It reports **non-missing** values. A count below the number of rows means missing data is present in that column.

`.groupby()` and `.agg()` describe each group

```python
import pandas as pd                                          # tables

asthma = pd.read_csv("data/pa_asthma_prevalence.csv")        # one row per county
counties = pd.read_csv("data/pa_counties.csv")               # metro status per county
joined = asthma.merge(counties, on=["county", "fips"])       # one table

summary = joined.groupby("metro").agg(                       # one row per metro group
    counties=("county", "count"),                            # how many counties in the group
    mean_asthma=("asthma_prev_pct", "mean"),                 # group mean of asthma
    mean_smoking=("smoking_prev_pct", "mean"),               # group mean of smoking
)                                                            # end of the named aggregations

print(summary.round(2))                                      # two decimals
```

```
          counties  mean_asthma  mean_smoking
metro
Metro           37        10.58         16.08
Nonmetro        30        10.75         18.42
```

The comparison this table makes

Metro and nonmetro counties differ by **0.17** percentage points in asthma prevalence, and by **2.34** percentage points in smoking prevalence.

`.corr()` measures how two columns move together

```
import pandas as pd                                          # tables

asthma = pd.read_csv("data/pa_asthma_prevalence.csv")        # one row per county
prev = asthma["asthma_prev_pct"]                             # the flat variable

print("smoking vs asthma:",                                 # label
      round(asthma["smoking_prev_pct"].corr(prev), 3))       # correlation, 3 decimals
print("obesity vs asthma:",                                 # label
      round(asthma["obesity_prev_pct"].corr(prev), 3))       # correlation, 3 decimals
```

```
smoking vs asthma: 0.659
obesity vs asthma: 0.476
```

A correlation runs from **−1** to **+1**. It reports the strength and direction of a **straight-line** relationship, and nothing else.

The answer to the opening question: asthma moves with smoking, at r = +0.659 across 67 counties.

What a correlation cannot establish

It cannot establish cause. It cannot detect a curved relationship. It is distorted by outliers. **And these are county averages, so it describes counties, not people.**

Session outcomes

1. The **figure/axes model**, and `plt.subplots()` as the entry point to every figure
2. matplotlib marks: `ax.scatter()`, `ax.hist()`, `ax.boxplot()`, and layers such as `ax.axvline()`
3. Pairs plots with `pd.plotting.scatter_matrix()`
4. The **grammar of graphics** in seaborn: data, mappings, and the mark as the function name
5. **Mapping versus setting**: `hue`, `style`, `size` against `color`, `marker`, `s`, `alpha`
6. Ordered categories with `pd.Categorical()`; labels with `ax.set()`; colours with `palette=`
7. **Choosing a figure** from the variable types; `countplot` for counts; **zero baselines for bars**
8. `.describe()`, grouped `.agg()`, and `.corr()` as co-movement, never cause

Preparation for the lab, Thursday 11/12

The lab builds each figure type from this session on the course datasets, and repairs a figure that misleads without raising an error.

Module 6 adds seaborn's statistical layers, facets, scales, and themes — and the question of when a figure of counts is really a figure of population.

That’s all folks!

See you on Thursday!